MATERIAL DIDÁCTICO · PMDM

TUTORIAL 01

Mi lista de compra en Android

APRENDER PROGRAMANDO, PASO A PASO

PROGRAMACIÓN MULTIMEDIA Y DISPOSITIVOS MÓVILES · 2.º DAM

TUTORIAL RESUELTO · LISTAS · ARRAYADAPTER · VALIDACIÓN

Objetivo y resultado

Construirás una aplicación que añade productos de texto a una lista, muestra cuántos hay y permite quitarlos tocando su fila. Harás el proyecto completo desde Android Studio, con Android Views (XML).

Preparación	Valor
Proyecto	ListaCompra
Plantilla	Empty Views Activity · Kotlin
Paquete	com.example.listacompra
Archivos	MainActivity.kt + activity_main.xml
Sin bibliotecas nuevas	Usaremos ListView y ArrayAdapter del entorno Android

Antes de tocar el código

1. Abre **Android Studio → New Project**. Elige la plantilla **Empty Views Activity** o equivalente con **activity_main.xml**. No selecciones Empty Activity con Compose.
2. Escribe **ListaCompra** y el paquete indicado. Deja el mínimo de SDK permitido en el aula.
3. Espera a que termine la sincronización de Gradle. Abre la vista **Project** y encuentra app/src/main/res/layout/activity_main.xml y MainActivity.kt.
4. Selecciona tu emulador. Ejecuta una vez la pantalla inicial para comprobar que el proyecto base funciona.

> **Comprueba:** puedes abrir la app sin errores. Si no existe activity_main.xml, has elegido una plantilla distinta: crea un proyecto Views.

1. Diseña la pantalla XML

En **res/layout/activity_main.xml** sustituye el diseño inicial por el siguiente. Es un LinearLayout vertical: arriba entrada y botón, debajo el estado y finalmente ListView. El **layout_weight="1"** ocupa el espacio restante.

Archivo: res/layout/activity_main.xml

XML / CÓDIGO COMPLETO O FRAGMENTO

```xml
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="20dp">

    <TextView
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Mi lista de compra"
        android:textSize="24sp" />

    <EditText
        android:id="@+id/etProducto"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Producto"
        android:inputType="textCapSentences" />

    <Button
        android:id="@+id/btnAgregar"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Añadir producto" />

    <TextView
        android:id="@+id/tvEstado"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Lista vacía"
        android:textSize="16sp" />

    <ListView
        android:id="@+id/lvProductos"
        android:layout_width="match_parent"
        android:layout_height="0dp"
        android:layout_weight="1" />
</LinearLayout>
```

Qué controles debes identificar

- **etProducto:** cuadro donde se escribe el artículo. **btnAgregar:** botón para insertarlo.
- **tvEstado:** informa del número de productos o del error.
- **lvProductos:** lista visual; no guarda los datos por sí misma.

> **Comprueba:** en la vista Design o Preview aparece una pantalla con campo, botón y zona de lista. Si el XML marca errores, revisa que solo haya una etiqueta raíz.

2. Programa la lista y los eventos

Abre **MainActivity.kt**. Sustituye su contenido por el código. Conserva el **package** que corresponda a tu proyecto; el ejemplo usa el nombre indicado al crearlo. No incluyas ninguna instrucción de impresión dentro de limpiarNombre().

Archivo: MainActivity.kt

KOTLIN / CÓDIGO COMPLETO O FRAGMENTO

```kotlin
package com.example.listacompra

import android.os.Bundle
import android.widget.ArrayAdapter
import android.widget.Button
import android.widget.EditText
import android.widget.ListView
import android.widget.TextView
import androidx.appcompat.app.AppCompatActivity

class MainActivity : AppCompatActivity() {
    private val productos = mutableListOf<String>()

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        val entrada = findViewById<EditText>(R.id.etProducto)
        val boton = findViewById<Button>(R.id.btnAgregar)
        val lista = findViewById<ListView>(R.id.lvProductos)
        val estado = findViewById<TextView>(R.id.tvEstado)
        val adaptador = ArrayAdapter(
            this, android.R.layout.simple_list_item_1, productos
        )
        lista.adapter = adaptador

        fun refrescar() {
            adaptador.notifyDataSetChanged()
            estado.text = "Productos: ${productos.size}"
        }

        boton.setOnClickListener {
            val producto = limpiarNombre(entrada.text.toString())
            if (producto == null) {
                estado.text = "Escribe un producto válido"
            } else {
                productos.add(producto)
                entrada.text.clear()
                refrescar()
            }
        }
        lista.setOnItemClickListener { _, _, posicion, _ ->
            productos.removeAt(posicion)
            refrescar()
        }
        refrescar()
    }

    private fun limpiarNombre(texto: String): String? {
        val limpio = texto.trim()
        return if (limpio.isEmpty()) null else limpio
    }
}
```

Lee el código por partes

1. La propiedad **productos** es una lista mutable: puede crecer o reducirse con cada acción.
2. **ArrayAdapter** conecta los Strings con una fila sencilla incluida en Android. La propiedad **lista.adapter** lo asigna a ListView.
3. **limpiarNombre()** recibe el texto y devuelve un nombre válido o null. No conoce los TextView ni el botón.
4. **btnAgregar** lee el texto, valida y añade un elemento; después **notifyDataSetChanged()** avisa de los cambios.
5. **setOnItemClickListener** obtiene la posición de la fila tocada y elimina ese elemento con **removeAt(posicion)**.

> **Estado en memoria:** al girar la pantalla o recrear la Activity se puede perder esta lista. En la UD3 aprenderás a guardar su estado o almacenarla en disco.

3. Ejecuta y comprueba

Prueba	Resultado esperado
Escribe Pan y pulsa Añadir	Aparece Pan; contador 1
Añade Leche y Fruta	Tres filas; contador 3
Pulsa Leche	Desaparece esa fila; contador 2
Campo vacío o solo espacios	No se añade nada; mensaje de error
Añade Avena después de borrar	Aparece como nueva fila; contador correcto

4. Comprende y amplía

- ¿Qué sucedería si no llamas a **notifyDataSetChanged()**? Compruébalo y vuelve a dejar el código correcto.
- Modifica el diseño para añadir un botón Limpiar todo que ejecute **productos.clear()** y actualice la pantalla.
- Ampliación opcional: impide nombres repetidos usando **any { it.equals(producto, true) }**.

> **Entrega del tutorial:** proyecto funcionando y dos capturas (lista de tres elementos y error de campo vacío). A continuación resuelve **UD2 · Relación 1**.